Final Report

Semester 5

ME3261 - Mechatronic System Design Project

Design and Development of vision based Human Following Robot for Super Market

By

Index No.	Name
200184T	Gayan K.A.K

Department of Mechanical Engineering
University of Moratuwa
Sri Lanka

19th December 2024

Table of Contents

List of figures

1 INTRODUCTION

Now days, supermarkets aim to enhance customer experience while optimizing operational efficiency. A human-following robot is designed to assist customers by carrying their items, reducing physical strain and improving convenience. This robot autonomously tracks and follows a customer, eliminating the need for traditional shopping carts.

Why Use a Human-Following Robot in Supermarkets?

- Customer Convenience**:** customers can focus on selecting products while the robot carries their items.
- Accessibility: It supports elderly or physically challenged customers, making shopping easier.
- Enhanced Shopping Experience: Personalized assistance creates a modern, tech-enabled shopping environment.

Key Features:

1. Autonomous Human Following: The robot uses advanced vision-based systems for real-time tracking.
2. Stable Mobility: A 6-wheel rocker-bogie design handles uneven surfaces and small steps.
3. Intelligent Control System: The robot responds to app-based commands, allowing versatile control.
4. Real-Time Processing: Image processing algorithm ensures accurate and adaptive human tracking.
5. Command Execution: The robot can respond to user commands such as moving forward, turning, and stopping. This functionality allows flexible and intuitive operation based on real-time inputs.

Combining computer vision, and mechatronics, the Human-Following Robot project provides an innovative shopping experience that benefits customers and businesses.

2 LITERATURE REVIEW

2.1 Research Based

2.1.1 Design of a Vision Based Person Following Robot

This paper introduces a person-following robot that can track a human using a webcam[1]. The system's entire operation can be split into two sections. The robot's hardware is the first component, followed by software, which detects objects, determines their position, and applies logic to provide control commands for the motor. Robot and human is always kept at a fixed distance from one another.

As in figure 2.1 to track the human they use red circle symbol, using the image processing technique identifying the radius of the circle and position of the circle in the frame using that information robot can identify the movement of the human.

Here they did not identify the human directly this is an indirect approach for human following, if there is any red circle other than this, some issues can occur.

Also, here there is no sensor for distance measurement, by taking the radius of the circle they get the distance. By comparing the circle radius, the robot determines the person's distance and then controls its speed to maintain a constant distance. When the target person moves forward, it moves forward, and when the person stops, the robot moves to a point behind the person and stops. If the person approaches the robot too much, it moves back.

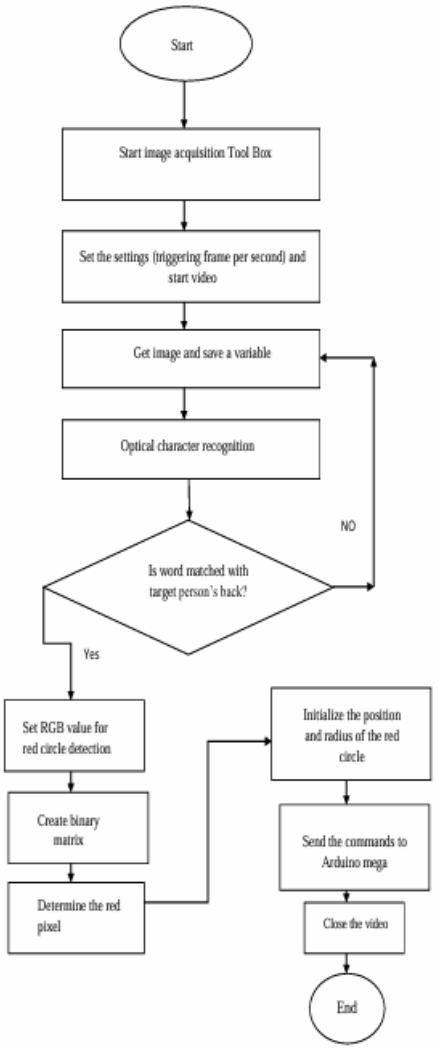


Figure 2.1: Flowchart of the program

Figure 2.2:Tracking Symbol

Figure 2.3:Practical Implementation

Figure 2.3 shows the practical implementation of the system.

2.1.2 Design and Development of Human Following Robot

The research focuses on designing and developing a human-following robot using a unique color-tag detection system. The robot uses a custom-designed tag with four distinct colors arranged in a specific pattern, enabling precise target recognition. A Raspberry Pi processes video input from a camera, while an Arduino-based control unit manages sensors, including ultrasonic, magnetometer, and infrared modules, for obstacle avoidance, distance maintenance, and orientation correction[2].

The system employs a decentralized approach with separate processing and control units, ensuring modular functionality and fault tolerance. Key features include intelligent tag detection, motor control through a relay-based driver, and distance measurement using ultrasonic sensors. Experiments validated the robot's ability to follow the target efficiently, even in challenging environments, while maintaining collision avoidance and tracking precision[2].

Here also they used custom tag to do the human following task which is an indirect method.

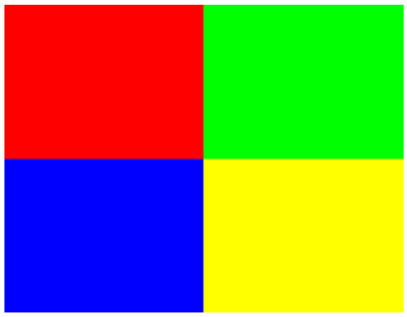


Figure 2.5:Custom Tag

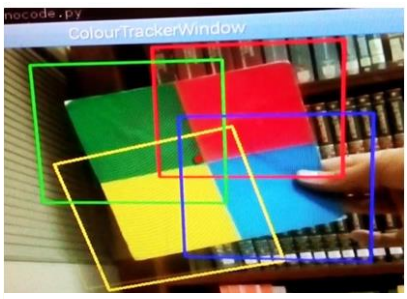


Figure 2.4:Tag Detection

Final prototype of the robot is shown in the following figure.

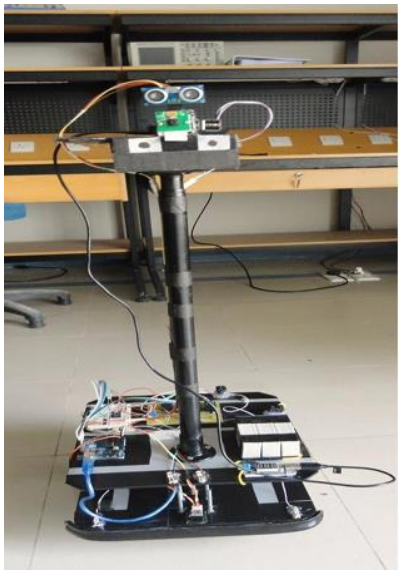


Figure 2.6:Human following Robot

2.1.3 Human following robot using image processing for medical application

The project includes combining image processing and sensor-based tracking to design a human-following robot for medical applications. By safely tracking and following patients in indoor medical environments, it aims to help patients[3].
Processing Unit: Raspberry Pi processes image data using OpenCV in Python.

Object Tracking: A Pi Camera detects a custom logo placed on the target person, guiding the robot's movement.

Sensors:

- Ultrasonic sensors for obstacle avoidance and distance maintenance.
- Three ultrasonic sensors are positioned (one central for distance tracking, two side sensors for obstacle detection).

Motor Control:

- SPG30-60 DC geared motors for movement.
- MDD10A dual-channel motor driver for motor control using PWM signals.

Power Supply:

- 12V lead-acid batteries for motor drivers and motors.

- A power bank supplies 5V to the Raspberry Pi.

Mechanical Design:

- Acrylic chassis with mounted motors and a free rear wheel for support.
- Adjustable camera height for optimal viewing.

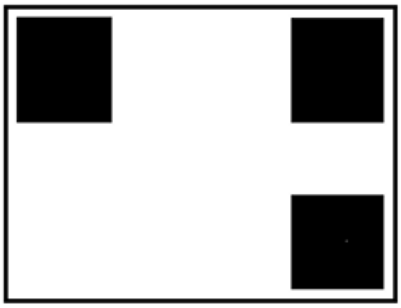


Figure 2.8:Logo of the tracking system

Figure 2.7:Final Setup

The Python code that controls the DC geared motor's direction via the motor driver is responsible for the robot's movement. In addition, the robot's movement direction to follow the intended individual is provided by the tracking location obtained from the image processing tracking[3].

Here also implemented the indirect approach to follow the human, they apply image processing to identify the logo not the human.

2.1.4 Leveraging Stereo-Camera Data for Real-Time Dynamic Obstacle Detection and Tracking

In this research they focused on Dynamic obstacle detection and tracking using complex and advanced image processing techniques. for dynamic obstacle detection and person tracking they used leverages stereo-camera data[4].

Image processing Steps:

- The stereo camera captures video frames processed by OpenCV.

- A disparity map is generated using block-matching or deep neural networks like MADNet for depth estimation.
- The system identifies moving objects using cluster-based segmentation (DBSCAN) and tracks them in 3D using centroid tracking.
- A 2D bounding-box detector, such as MobileNet-SSD, classifies humans and assigns tracking IDs.
- A Kalman filter estimates target velocities, enabling dynamic object classification and future path prediction.

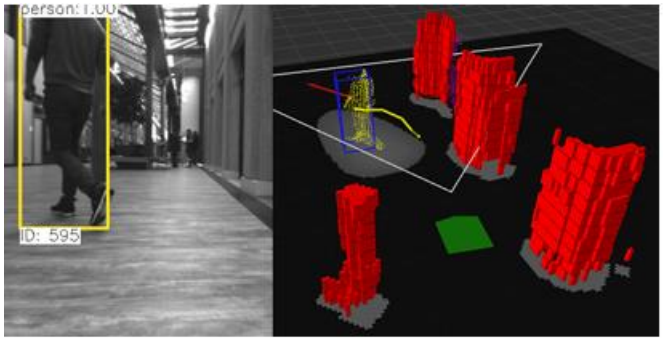


Figure 2.9:2D Bounding Box & 3D segmentation

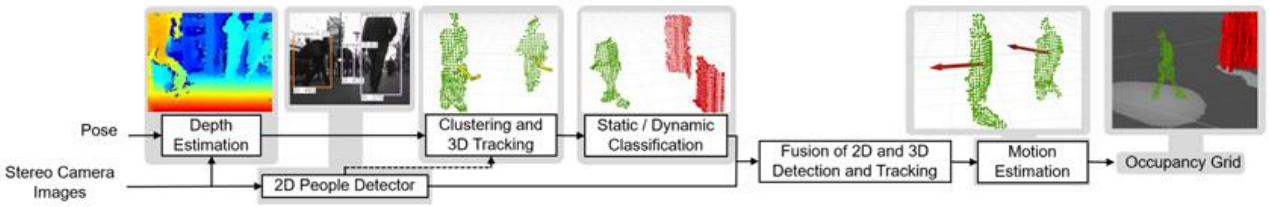


Figure 2.10:Overview of the pipeline

This is suitable for Human following task, but the issue is complexity of the algorithm, we need high processing power controller to use this.

2.2 Commercially available Systems

2.2.1 Segway Robotics Loomo

The Segway Loomo is a smart personal robot that integrates mobility and artificial intelligence. It has capabilities such as obstacle avoidance, voice and gesture commands, and autonomous following.

Key features:

- Autonomous Following: Using modern computer vision, Loomo follows a user on their own while capturing steady-state footage.

- Voice and Gesture Controls: Loomo's five-microphone array allows it to recognise movements and voice instructions, facilitating natural user interaction.
- Obstacle Avoidance: By integrating the Intel RealSense ZR300 camera, Loomo is able to detect and avoid obstacles in real time by detecting depth.
- Self-Balancing Personal Transporter: With a range of up to 22 miles on a single charge, the Loomo provides a smooth ride over a variety of terrains.

Figure 2.12:Loomo Robot

Figure 2.11:Object detection

From https://www.segway.com/loomo/

2.2.2 Aido Robot

Aido is an advanced social family robot developed by InGen Dynamics, designed to assist with various household tasks and provide entertainment.

Aido include voice recognition, Gesture control and Autonomous movement. It navigates using a multi-sensor fusion approach, avoiding obstacles and mapping its environment. Also, it has advanced computer vision algorithms that enable facial and object recognition and cameras to track and identify familiar faces, supporting personalized interactions. It has Quad-Core ARM7 CPUs and integrated GPUs for efficient computing processer.

From https://aidorobot.com

Figure 2.13:Aido Robot

3 GEOMETRY ANALYSIS

3.1 Design Parameters

Total Mass with pay load = 10kg

Max step height = 5cm

Trackwidth = 50cm

Wheelbase = 52.5cm

Wheel diameter = 12cm

Tire Material: Rubber (Pure rolling wheels)

3.1.1 Bogie Dimensions

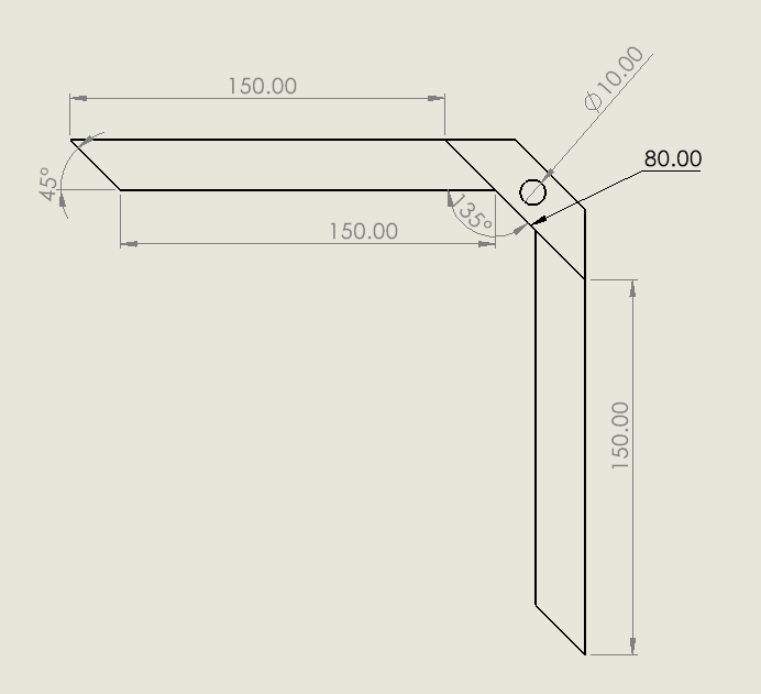


Figure 3.1:bogie dimensions

Dimensions in mm.

3.1.2 Rocker Dimensions

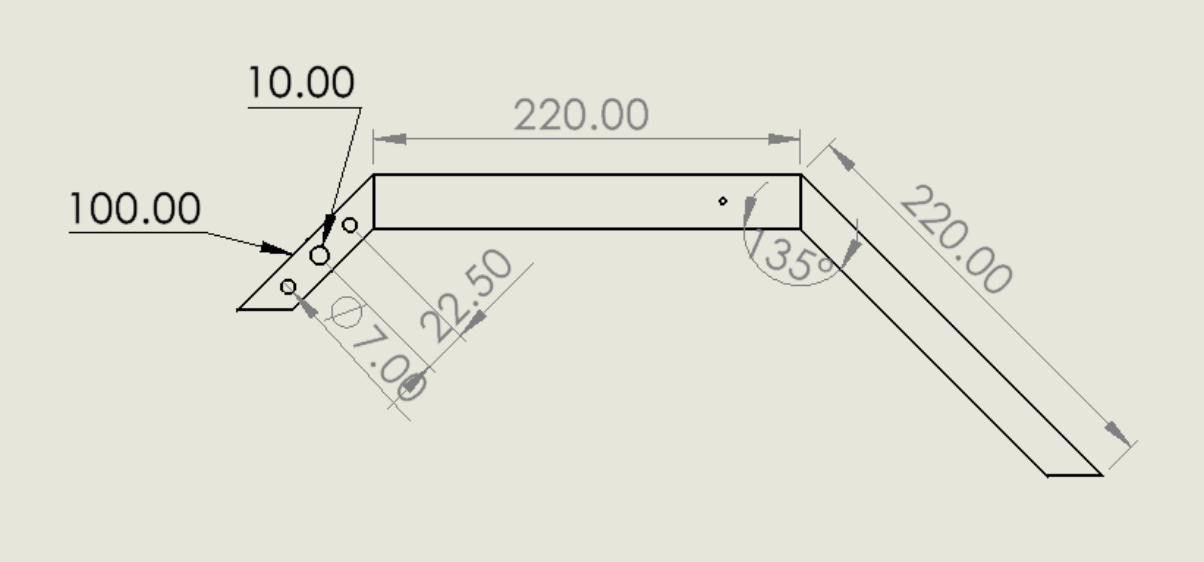
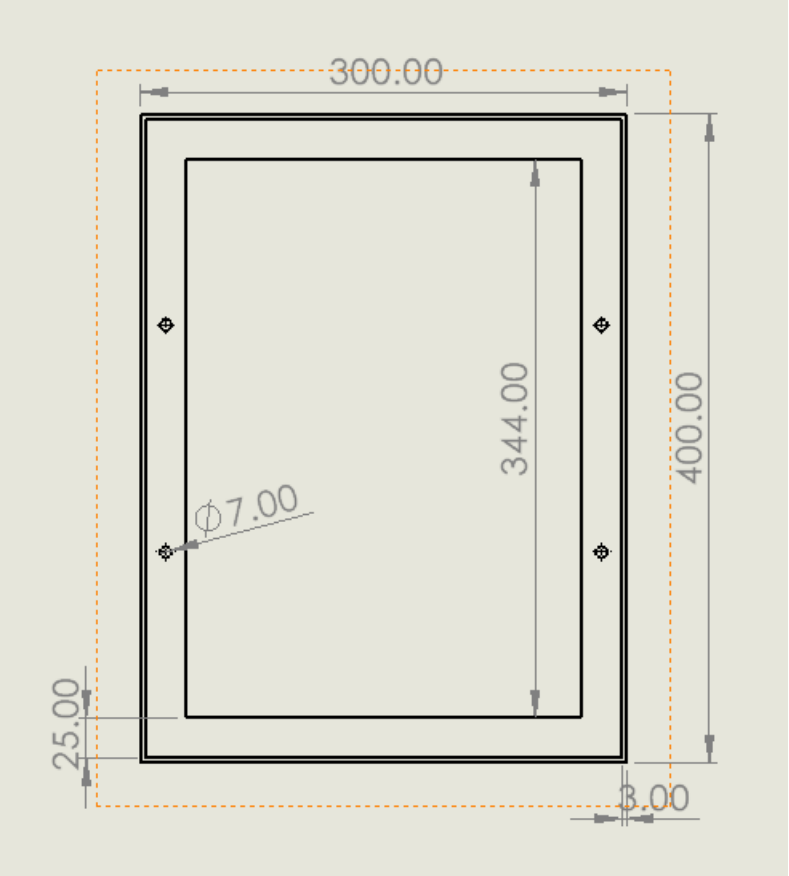


Figure 3.2:rocker dimensions

Dimensions in mm.

3.1.3 Chassie Dimensions



Dimensions in mm

Figure 3.3:chassie dimensions

3.2 Links Dimensions Analysis

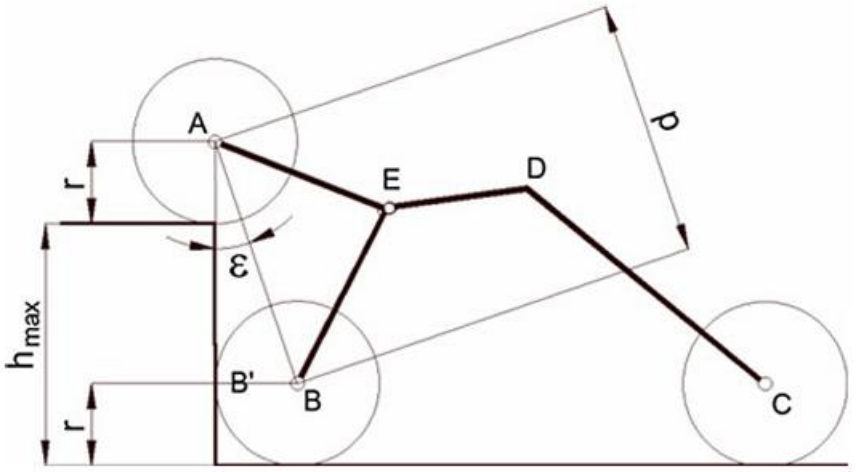


Figure 3.4:Theoritical Maximum height

The geometric theoretical maximum height of the obstacle step that a rover can overcome is defined as the one for which wheel A is on the obstacle, with wheel B at the beginning of the climbing phase[5].

$$h_{max} = p\cos\varepsilon$$

$$\varepsilon = \arcsin\left(\frac{r}{p}\right)$$

Where $p = \sqrt{2} \times bogie\ link\ length$

According to the design parameters we can calculate h_{max} as follows.

$$p = \sqrt{2} \times bogie\ link\ length$$

$$p = \sqrt{2} \times 15$$

$$\varepsilon = \arcsin\left(\frac{r}{p}\right)$$

$$\varepsilon = \arcsin\left(\frac{6}{\sqrt{2} \times 15}\right)$$

$$h_{max} = \sqrt{2} \times 15 \cos\left(\arcsin\left(\frac{6}{\sqrt{2}\times 15}\right)\right) = 20.35\text{cm}$$

According to the calculations links dimensions are suitable for our application. Also, in here we only consider about the geometrical parameters but when it come to practical scenarios friction, payload, Motor torques, Reaction forces. Etc likewise different factors is affected. Due to that maximum height that can be overcome by the robot may be changed.

4 STRUCTURAL ANALYSIS

4.1 3D model

Using the SolidWorks design each component of the robot. Material is Aluminum.

4.1.1 Bogie part

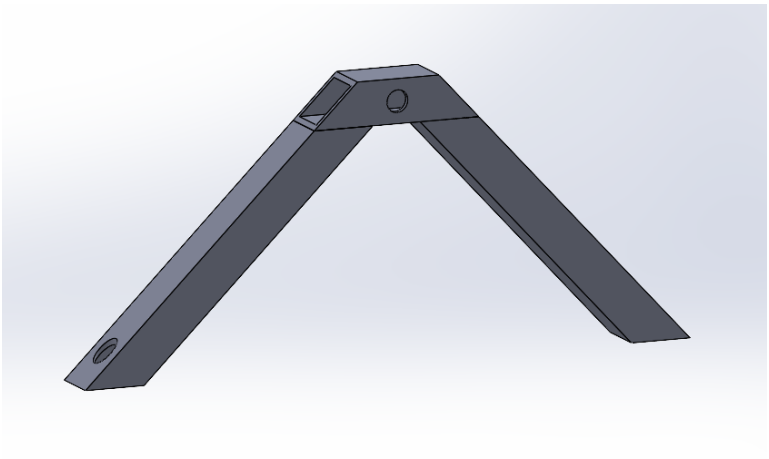


Figure 4.1:bogie part

4.1.2 Rocker part

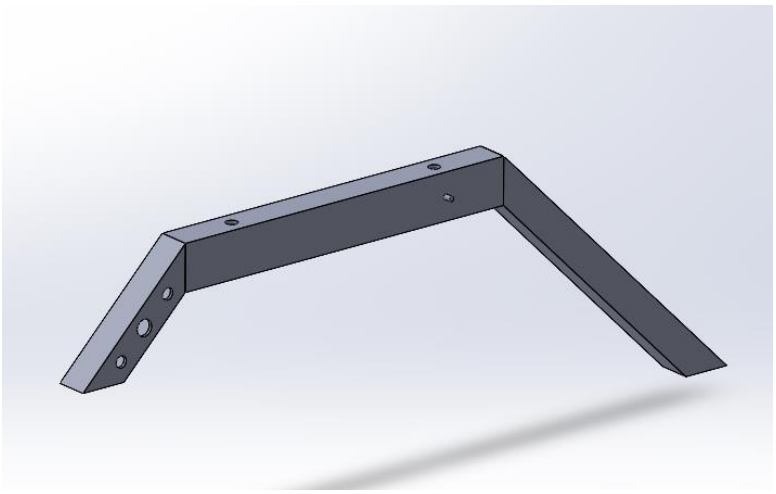


Figure 4.2:rocker part

4.1.3 Chassie frame

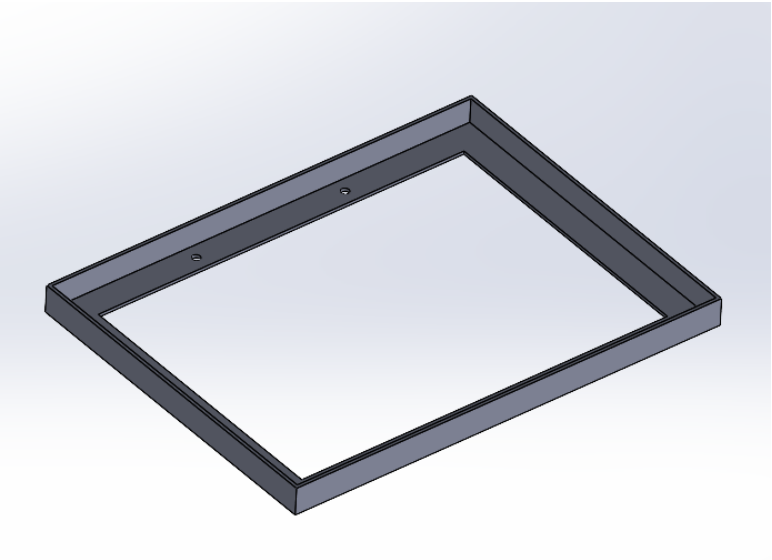


Figure 4.3:chassie frame

4.2 MESH

SolidWorks assembly file is converted to IGS format and import to Ansys software to do the static structural analysis[6]. Following figures shows the mesh, stress, strain, deformation of the structures.

4.2.1 Bogie part

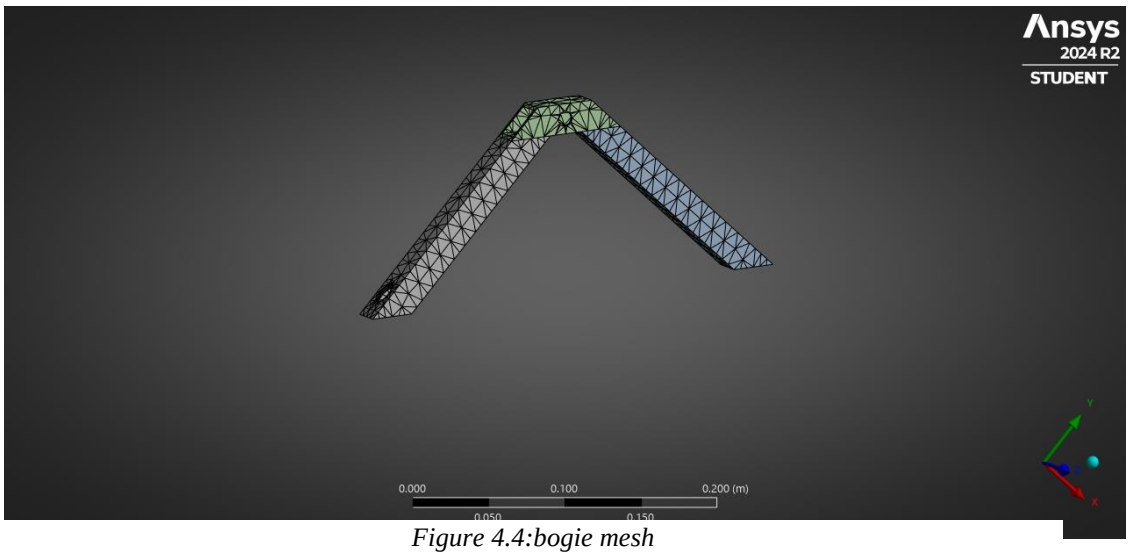


Figure 4.4:bogie mesh

4.2.2 Rocker part

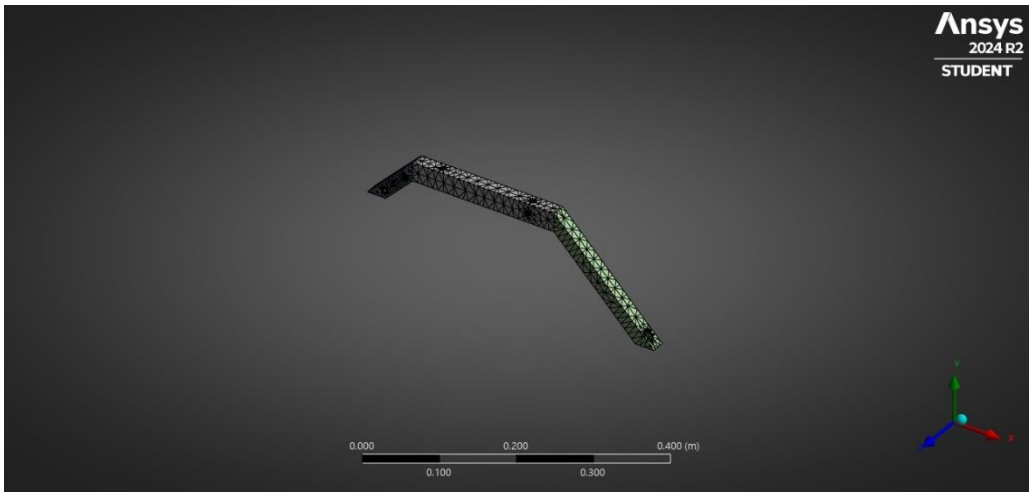


Figure 4.5:rocker mesh

4.2.3 Chassie frame

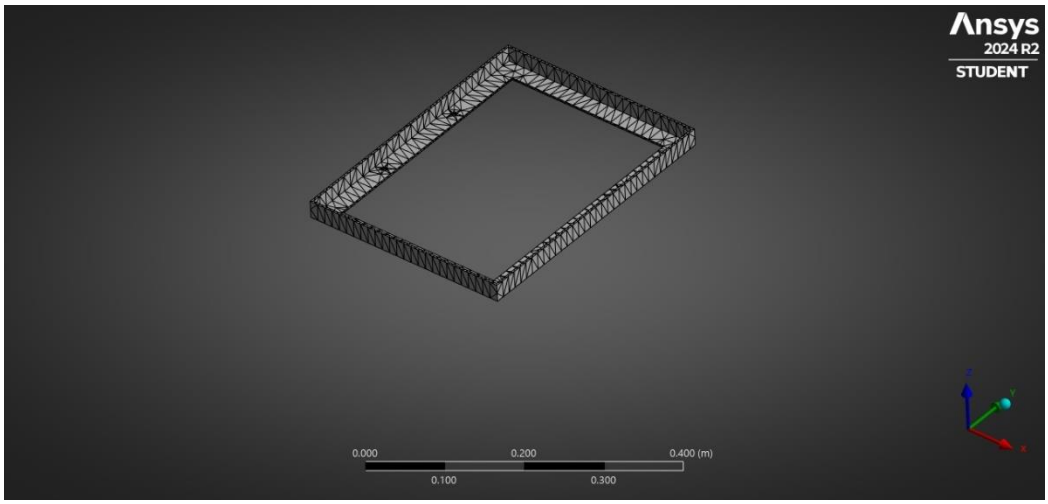


Figure 4.6:chassie frame mesh

4.3 Stress Distribution Results

By analysing static nodal stresses, identify the potential weak points in a structure that might be prone to failure under load

4.3.1 Bogie part

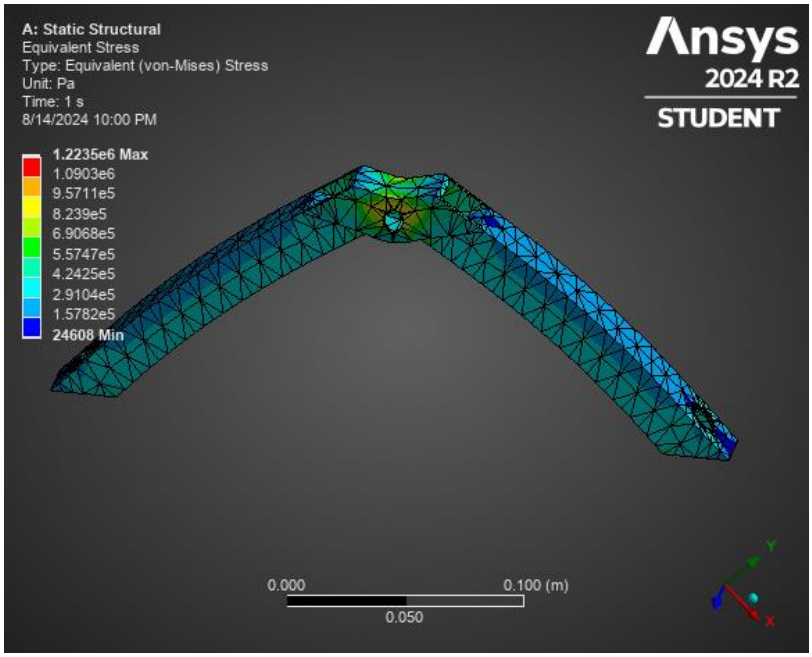


Figure 4.7:bogie stress distribution

4.3.2 Rocker part

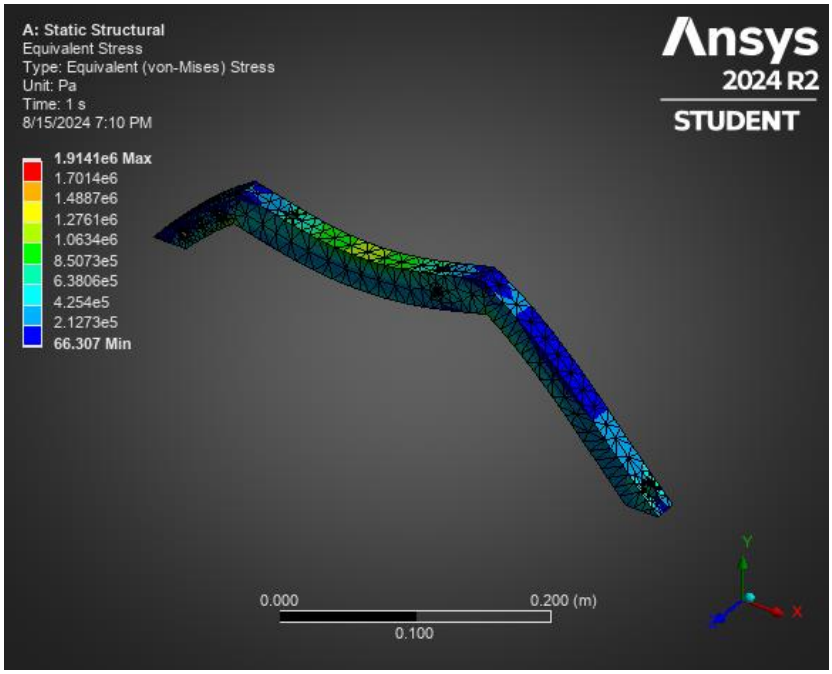


Figure 4.8:rocker stress distribution

4.3.3 Chassie frame distribution

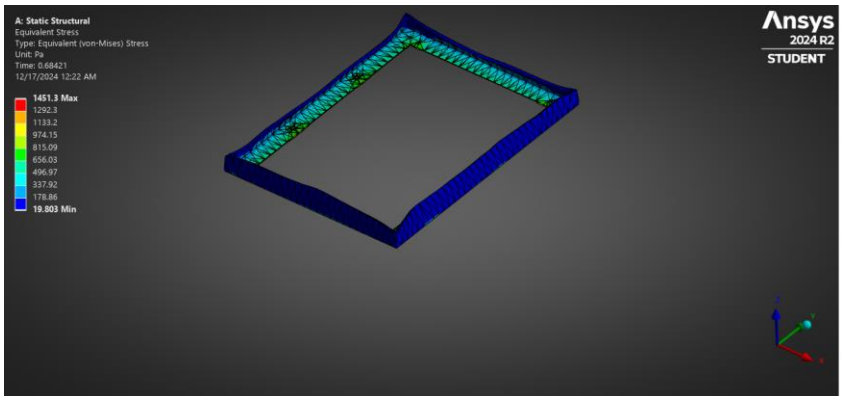


Figure 4.9:chassie frame distribution

4.4 Total Deformation

4.4.1 Bogie part

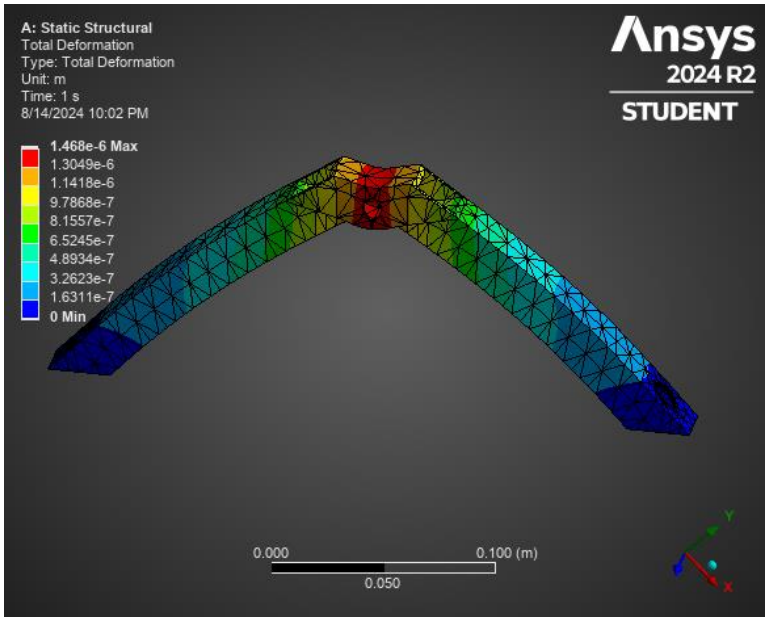


Figure 4.10:bogie deformation

4.4.2 Rocker part

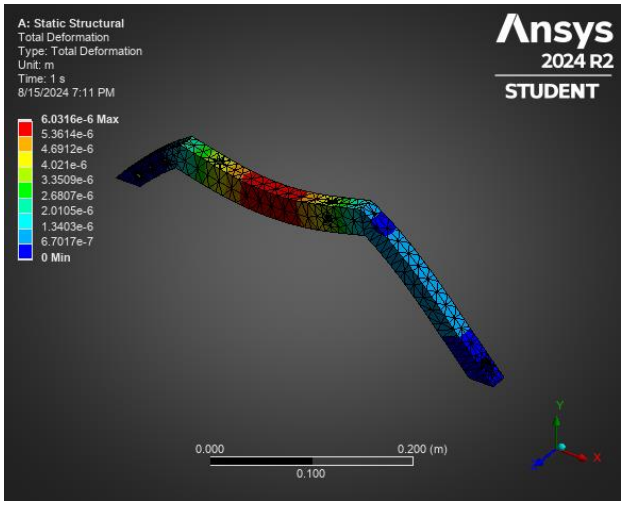


Figure 4.11:rocker deformation

4.4.3 Chassie frame part

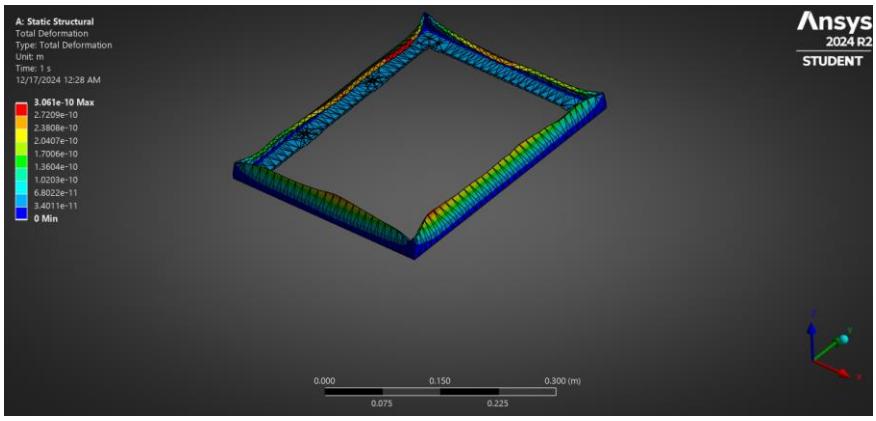


Figure 4.12:chassie frame deformation

4.5 Final Design of the Robot

Figure 4.13:Final Design

5 MOTION ANALYSIS

Using SolidWorks Motion study analyse the stability of the robot and the Yaw, Pitch, Raw and these graphs are plotting here.

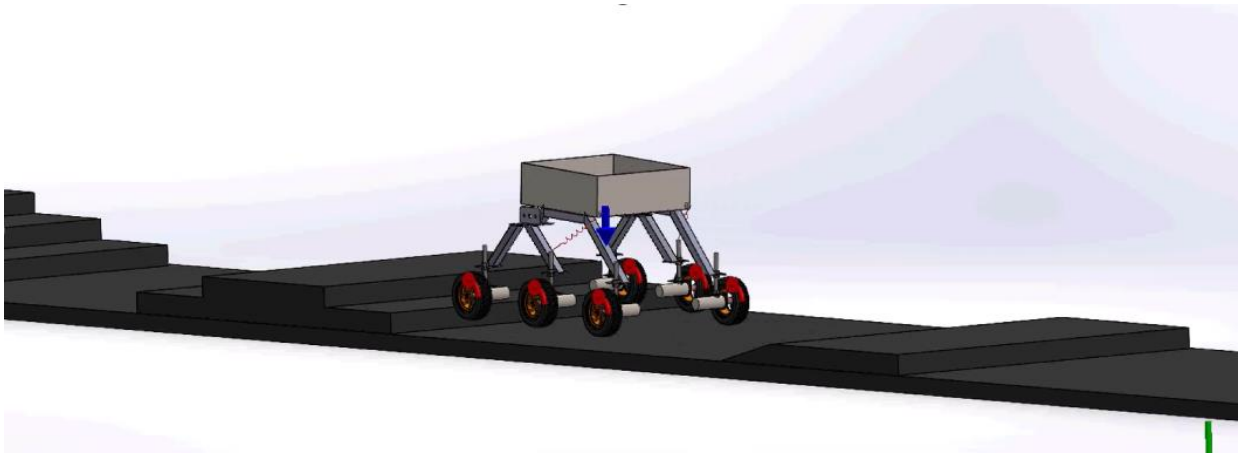


Figure 5.3:positioning robot in SolidWorks motion study

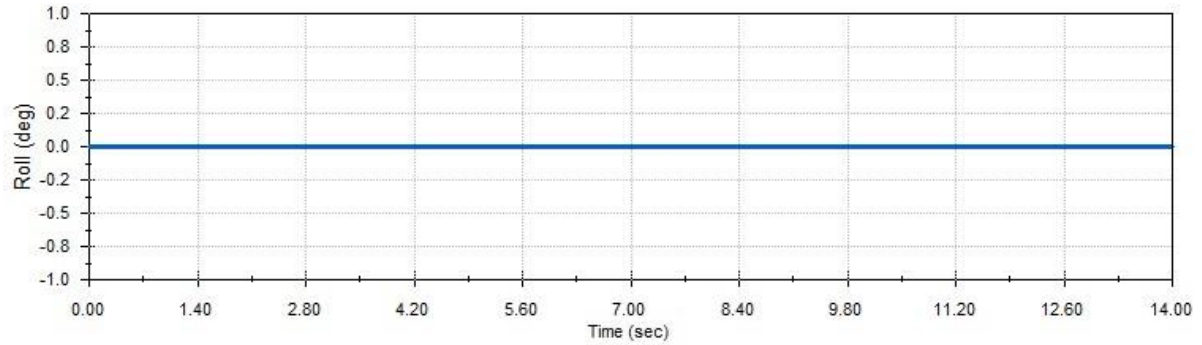


Figure 5.2:rolling

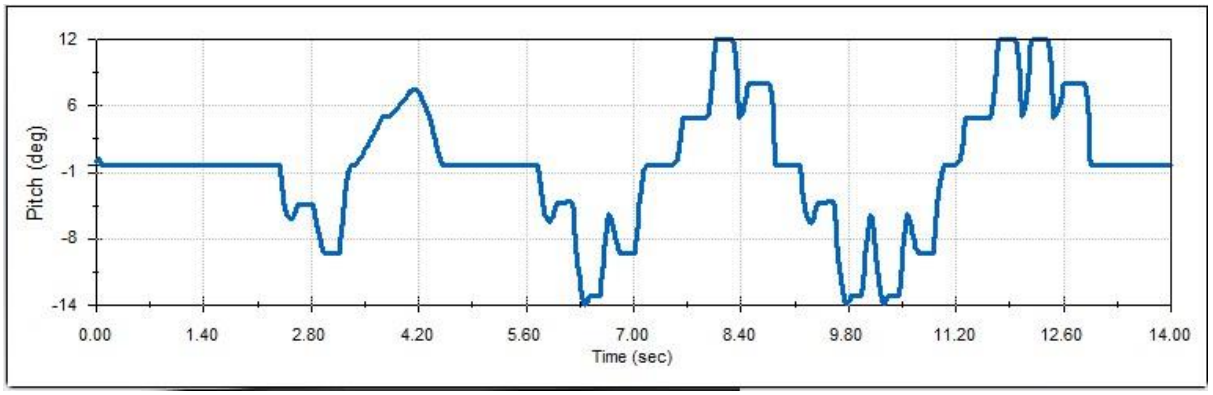


Figure 5.1: pitching

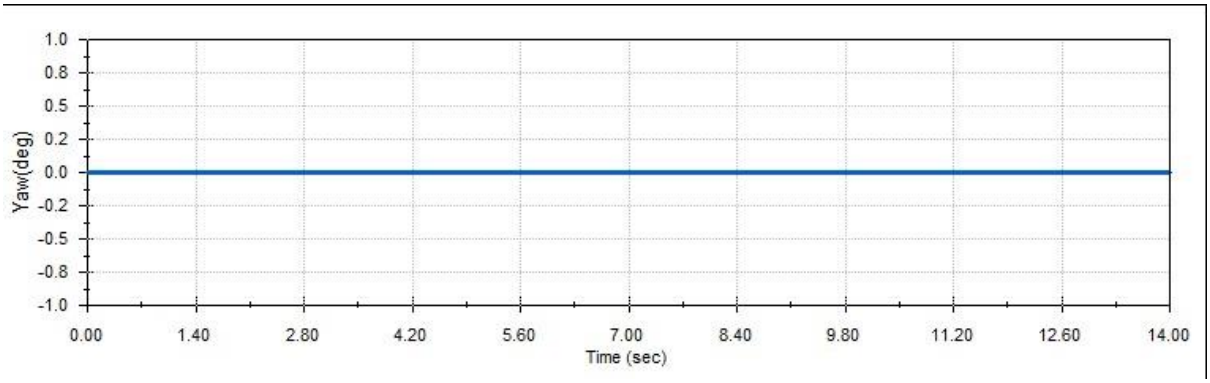


Figure 5.4: Yaw

After that according to the step climbing task pitching graph plotting the results.

6 KINEMATICS ANALYSIS

6.1 Steering Method

Skid-steer point-turn configuration is ideal for human-following robots due to its ability to change direction quickly with minimal time and space requirements. Here's why it is suitable:

- Zero Turning Radius:

  Skid-steer robots have a zero turning radius because they can rotate on their central axis. This reduces the need for extra operating area and enables the robot to change direction quickly.
- Quick Direction Change:

  By driving the left and right wheels in opposite directions, the robot can pivot instantly, making it highly responsive to sudden directional changes needed in human-following tasks.
- Reduced Turning Time:

  Since the robot can align its front to the detected person rapidly, minimizes downtime when switching directions, helping the robot maintain a close distance to the target person.

6.1.1 Kinematic Model Overview

Robot's motion can be described using its position and orientation in a 2D plane. The robot's position is given by coordinates (x, y), and its orientation (heading) is denoted by ψ, which is the angle the robot makes with the positive x-axis[7].

- State vector of robot in 2D plane

$$\eta = \begin{bmatrix} x \\ y \\ \psi \end{bmatrix}$$

x, y: Robot's position in the global coordinate system

ψ: Orientation (heading angle) of the robot.

- Wheel angular velocity vector

  w = Transpose [w_1, w_2, w_3, w_4, w_5, w_6]

Using this wheel angular velocity matrix we can represent our motor rpm value. after that these angular velocities should be converted to linear velocity of the robot, to do that we need configuration matrix.

- Configuration matrix

$$W = \begin{bmatrix} \frac{a}{2} & \frac{a}{2} & \frac{a}{2} & \frac{a}{2} & \frac{a}{2} & \frac{a}{2} \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -\left(\frac{a}{2d}\right) & \left(\frac{a}{2d}\right) & -\left(\frac{a}{2d}\right) & \left(\frac{a}{2d}\right) & -\left(\frac{a}{2d}\right) & \left(\frac{a}{2d}\right) \end{bmatrix}$$

a: radius of the wheel

d: Half of the trackwidth

- Robot's generalized velocity

$$\zeta = W.w$$

  W: Configuration matrix

  w: vector of wheel angular velocity

Now we have robot's generalized velocity with respect to the robot coordinate system, we have to transform that into global coordinate system.

This transformation is achieved using the Jacobian matrix $J(\psi)$.

$\dot{\eta} = J(\psi).\ \zeta$

$$J(\psi) = \begin{bmatrix} cos(\psi) & -sin(\psi) & 0 \\ sin(\psi) & cos(\psi) & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- Euler Method for Position Update

The robot's position and orientation at each time step are updated using Euler's method, which integrates the velocity over time:

$$\eta_{i+1} = \eta_i + \Delta t . \dot{\eta}_i$$

η_i: robot's state at time step i.

$\dot{\eta}_i$: Velocity at time step i.

Δt: time step size.

For each variable we can represent it as below.

$$x_{new} = x_{current} + vcos(\theta)\Delta t$$

$$y_{new} = y_{current} + vsin(\theta)\Delta t$$

$$\theta_{new} = \theta_{current} + \omega\Delta t$$

Where: (x, y): Current robot position in the plane.

θ: Current orientation (heading angle).

v: Linear velocity.

ω: Angular velocity.

Δ t: Time step.

These updates occur at each time step, giving a continuous estimate of the robot's new position and heading.

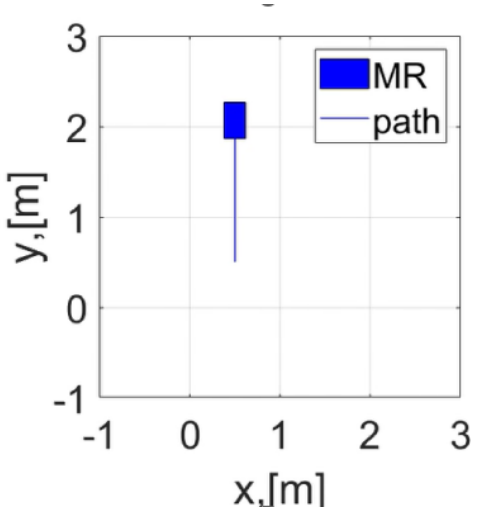


Figure 6.1:MATLAB forward motion result

Forward motion results obtained from the MATLAB simulation. the following figure shows the turning angle variation with respect to the time.

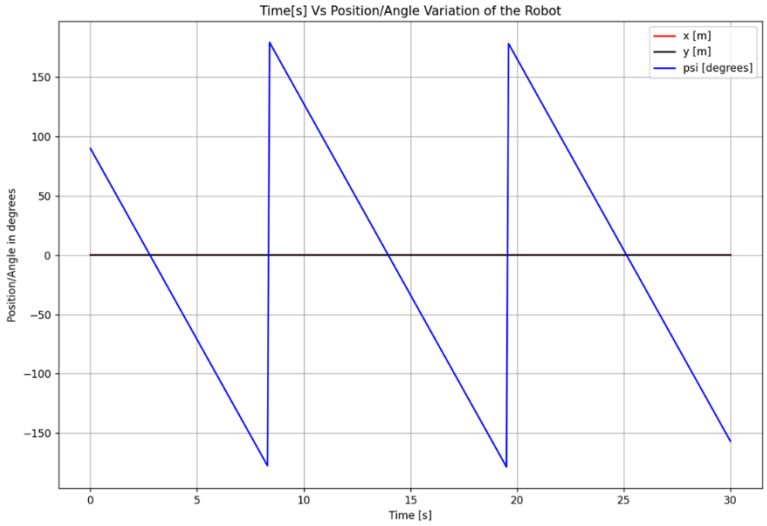


Figure 6.2: Turning angle variation with time

7 DYNAMIC ANALYSIS

Using Adam software simulate the robot and find the torque of the motors

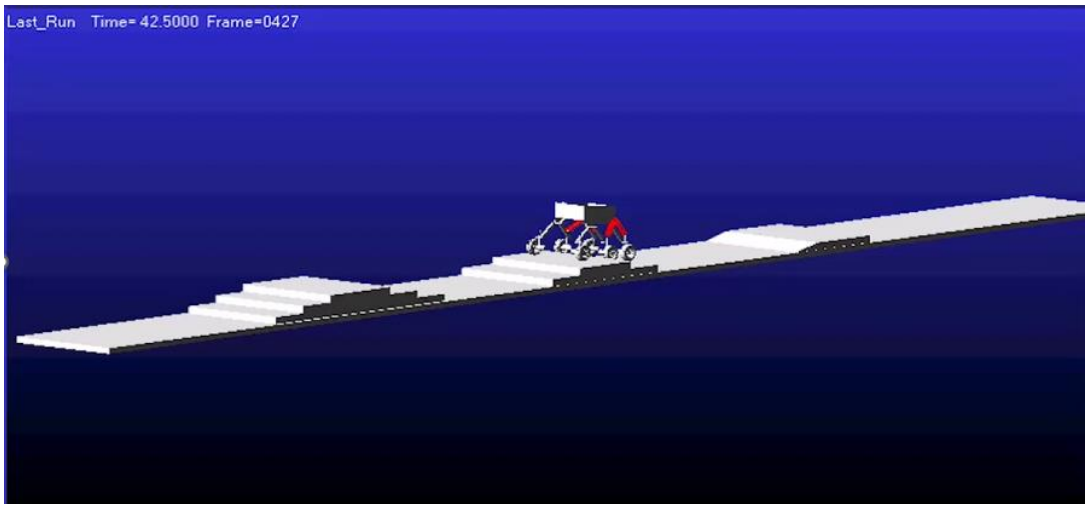


Figure 7.1:Adams simulation

7.1 Front left wheel torque distribution

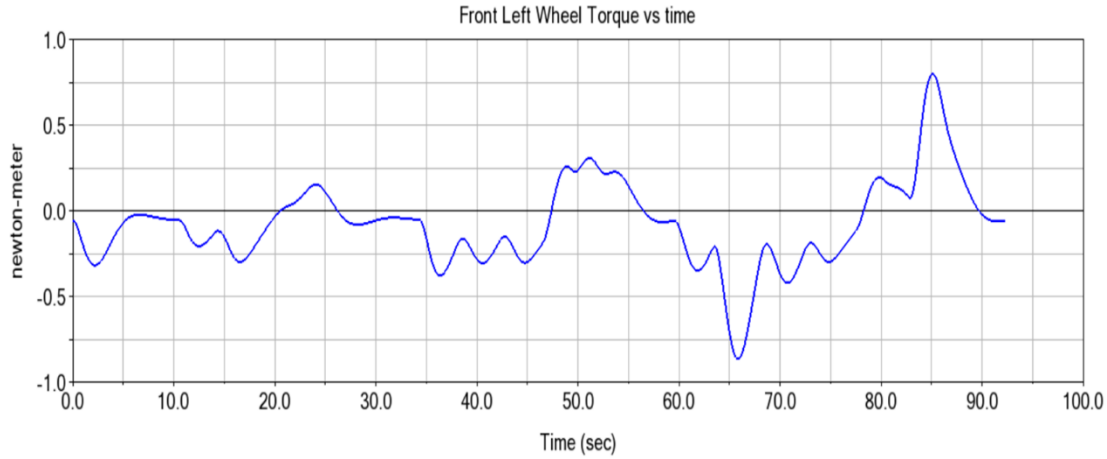


Figure 7.2:front left wheel torque

7.2 Middle left wheel torque distribution

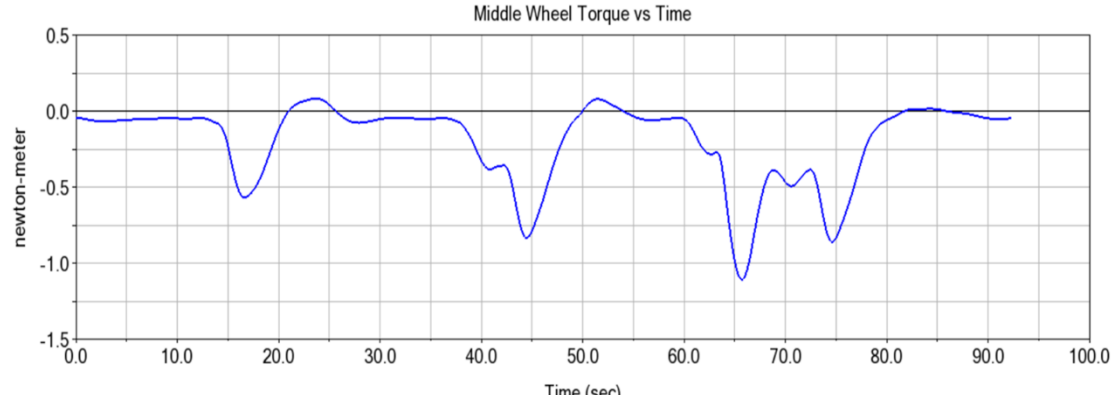


Figure 7.3:Middle left wheel torque

7.3 Back left wheel torque distribution

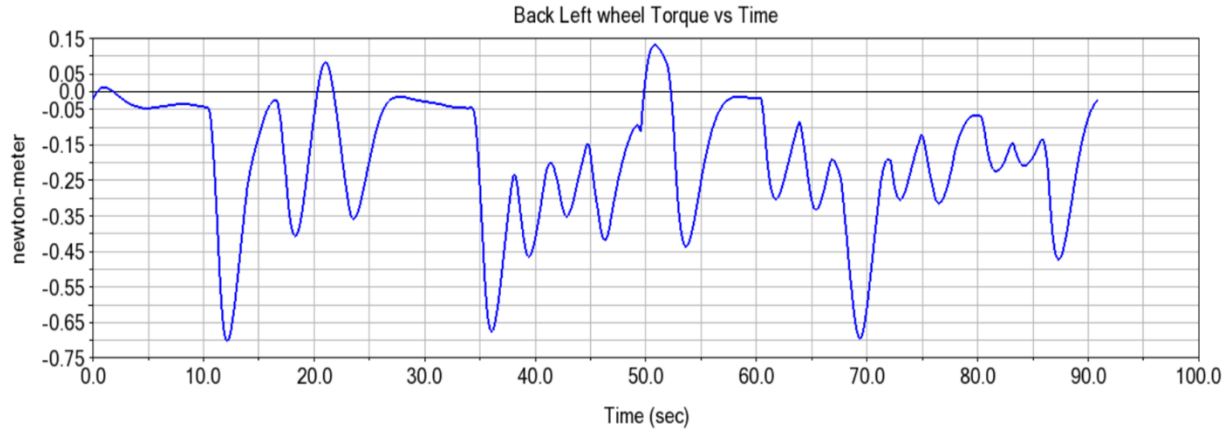


Figure 7.4:back left wheel torque

7.4 Suspension force variation

To increase stability, traction, and adaptation over uneven terrain, a rocker-bogie robot uses a shock absorber. The robot's wheels may lose contact with the ground during load shifts taken place during turning and step climbing, which could result in slipping. By allowing independent wheel movement, a spring helps in load redistribution, maintaining continuous ground contact and improving traction. Additionally, it reduces motor strain and improves overall stability by absorbing shocks from obstacles.

Stiffness coefficient :6.9Nm,

damping coefficient :1.7Ns/m

Force variation of the shock absorber

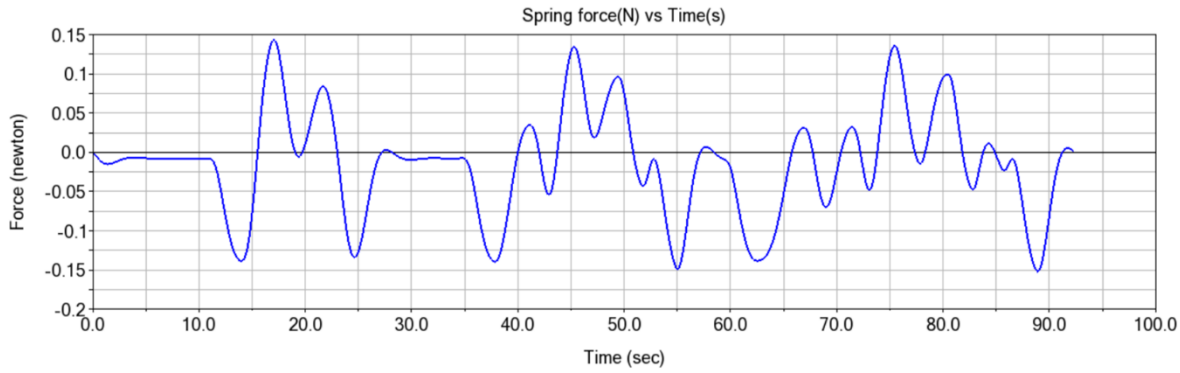


Figure 7.5: spring force variation

7.5 Motor Selection

According to the Adams result maximum torque required is 1 Nm, according to that following motor is selected for the application.

- XD-37GB520 DC Gear Motor

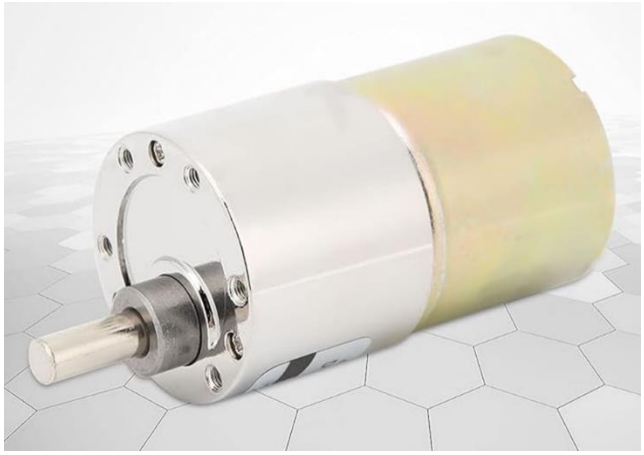


Figure 7.6:gear motor

8 IMAGE PROCESSING

8.1 Person Detection Methods

8.1.1 Traditional Image Processing (e.g., OpenCV)

Traditional image processing using OpenCV involves applying computer vision techniques to detect and track objects, including humans, without relying on deep learning models. Common methods include:

- background subtraction,
- edge detection,
- contour detection
- thresholding.

For example, detecting a person might involve applying background subtraction to isolate moving objects from a static background, followed by contour detection to identify the person's shape. These methods are computationally lightweight and suitable for simpler tasks or environments with controlled lighting and minimal background clutter[8].

In dynamic situations, however, conventional image processing is limited. It has trouble with variations in shadows, and illumination, which can lead to missed targets or false detections. In contrast to deep learning models, OpenCV-based methods require manual feature extraction and algorithm development because they do not have built-in human posture estimation. Compared with modern deep learning-based techniques, they are less flexible for complicated real-world scenarios since they require precise adjustment of parameters like thresholds and filter sizes, even though they are efficient for simple tasks.

8.1.2 MediaPipe Based Approach

The MediaPipe-based approach leverages advanced machine learning models to detect and track humans by estimating body landmarks in real time. MediaPipe uses a pipeline of pre-trained models that process video frames to identify key points such as shoulders, elbows, and knees, forming a skeletal representation of the person. This approach is particularly useful for tasks like pose estimation, human-following, and activity recognition, as it provides detailed spatial information about a person's posture and movement. MediaPipe is optimized for efficiency, running on CPUs and GPUs, making it suitable for real-time applications[9].

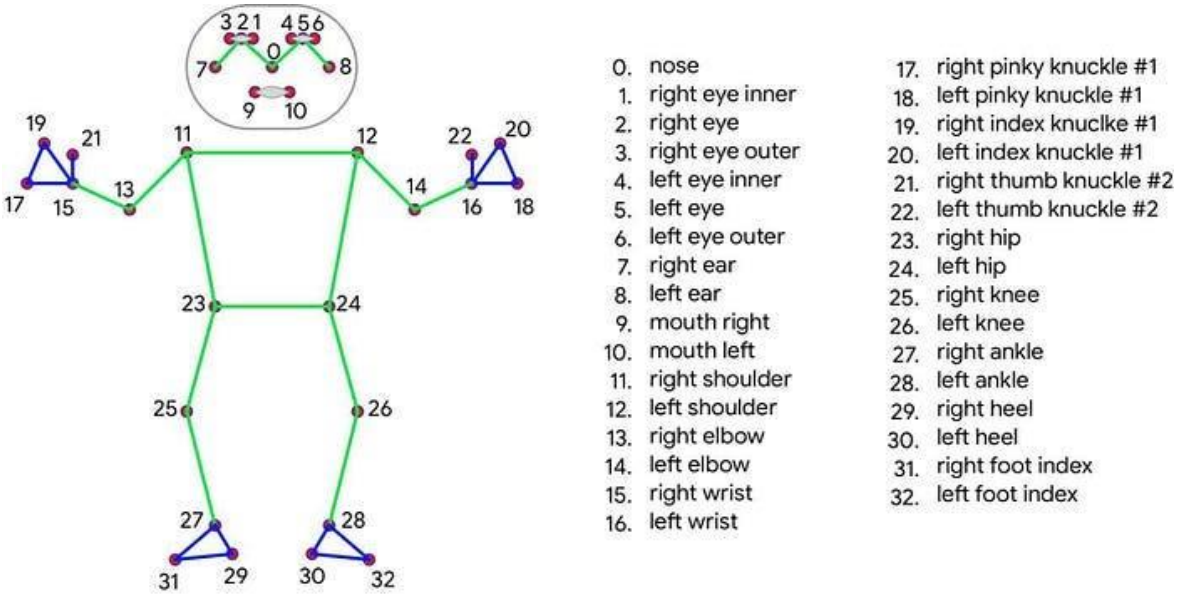


Figure 8.1:mediapipe pose estimation

MediaPipe's main strength is its capacity to handle complex scenarios with different people and dynamic environments. It reduces the complexity of development because it does not involve human feature extraction or parameter tuning, compared to traditional image processing. MediaPipe's models operate reliably even in difficult situations since they are

resistant to changes in illumination and perspective. Its real-time processing, built-in pose estimation, and cross-platform compatibility make it an ideal choice for projects like human-following robot.

8.1.3 Deep learning-based methods

Deep learning-based methods for human detection and tracking use artificial neural networks trained on large datasets to identify and classify people in images or videos. Models like YOLO (You Only Look Once), SSD (Single Shot MultiBox Detector), and ResNet (Residual Networks) are popular choices. These models process input frames through convolutional neural networks (CNNs) that automatically extract features such as edges, textures, and object shapes. They output bounding boxes, labels, and confidence scores, enabling accurate detection of people and other objects.

Deep learning techniques' main advantages are their high accuracy and robustness in complex environments. They can handle multiple people, different poses, varying lighting conditions.

How ever these models require high processing power, in real-time scenarios GPU is required to process these models without delay.

8.2 Image processing approach in Project

In the project, mediapipe based person detection method is used because it is suitable for operating with CPU.

8.2.1 Making grid

video frame is divided into nine equal segments using OpenCV, the frame’s dimensions are first extracted, and the width and height of each segment are calculated by dividing the frame's width and height by three. Vertical and horizontal lines are drawn using cv2.line(), creating a 3x3 grid. The centre segment is identified using its coordinates based on the second row and second column indices. To highlight this region, cv2.rectangle() is used to draw a rectangle with a distinct colour.

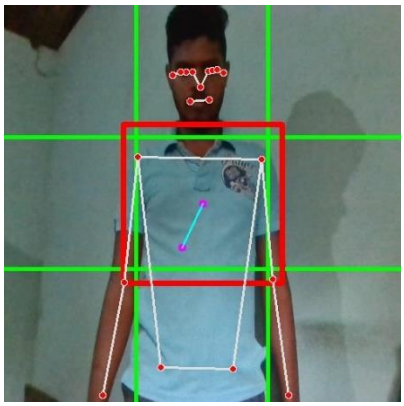


Figure 8.2:3x3 grid

8.2.2 Person detection from front side and backside

Using mediapipe pose estimation library identify the person.

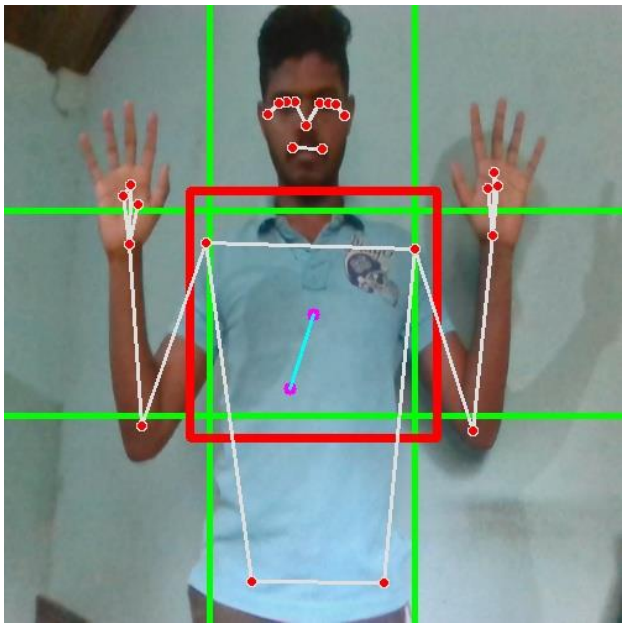


Figure 8.3:front side person detection

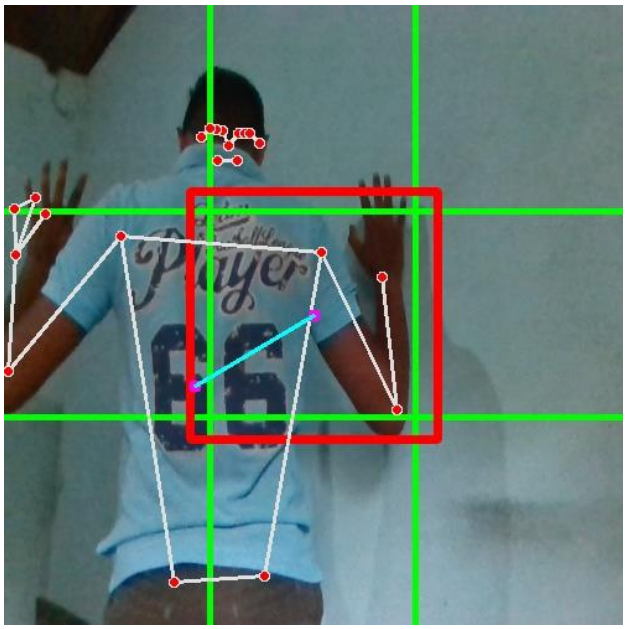


Figure 8.4:backside person detection

From the frontside and backside person was successfully detected, and all the landmarks are located into the detected person as in the figures.

8.2.3 Tracking the detected person

To track the person, get the person (x, y) coordinate like below.

$$x_{shoulder\ middle} = \frac{(x_{left\ shoulder} + x_{right\ shoulder})}{2}$$

$$y_{shoulder\ middle} = \frac{(y_{left\ shoulder} + y_{right\ shoulder})}{2}$$

$$x_{hip\ middle} = \frac{(x_{left\ hip} + x_{right\ hip})}{2}$$

$$y_{hip\ middle} = \frac{(y_{left\ hip} + y_{right\ hip})}{2}$$

$$x_{body\ centre} = \frac{(x_{shoulder\ middle} + x_{hip\ middle})}{2}$$

$$y_{body\ centre} = \frac{(y_{shoulder\ middle} + y_{hip\ middle})}{2}$$

Rather than using one point here I take average of 4 landmarks points in the body to track the person. After that using cv2.line() command draw the line by matching points of ($x_{body\ centre}$, $y_{body\ centre}$) and ($x_{frame\ centre}$, $y_{frame\ centre}$).

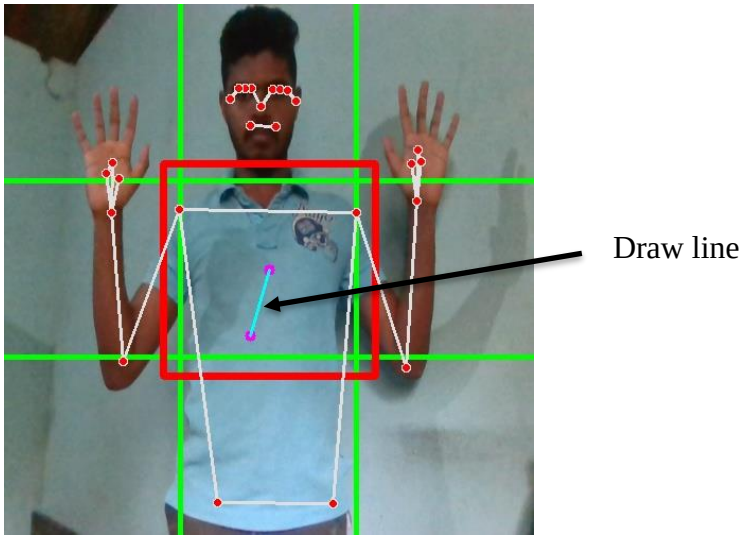


Figure 8.5:person tracking

According to the person body centre coordinate position robot will move forward, backward and steer.

9 GRAPHICAL USER INTERFACE FOR CONTROLLING

For controlling purpose graphical user interface is developed using Custom-tkinter library in python[10].

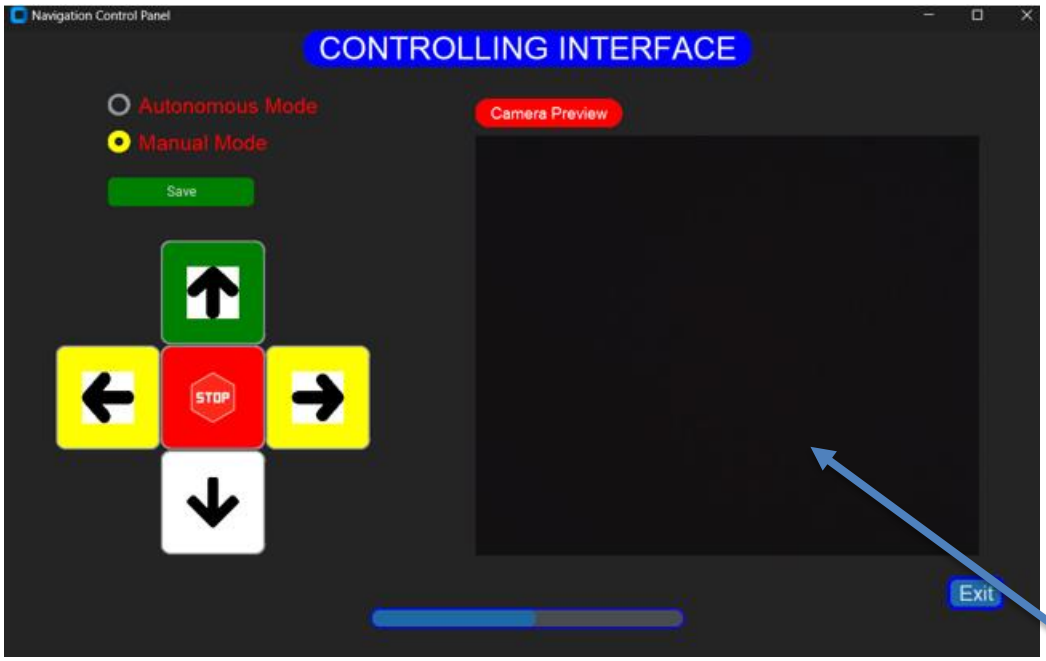


Figure 9.1:graphical user interface

In this graphical user interface, there are two control methods.

- Manual Mode
- Autonomous Mode

9.1 Manual Mode

In Manual Mode, the robot's movement is directly controlled by the user through a **Graphical User Interface (GUI)** equipped with arrow key buttons. These buttons allow the user to send specific motion commands to the robot, enabling it to move in different directions and execute turns. Each arrow key corresponds to a predefined control signal, which dictates the behaviour of the robot's motors.

video feedback window is integrated into the Graphical User Interface (GUI) to provide the user with a real-time visual feed of the environment in front of the robot. This video stream is typically captured using a camera mounted on the front of the robot and displayed directly

within the GUI. The inclusion of this video feedback serves as a crucial enhancement for the user, enabling them to monitor the robot's surroundings and make informed control decisions.

9.2 Autonomous Mode

In Autonomous Mode**,** an image processing algorithm is executed to detect and track a person in the robot's field of view. The system uses a camera mounted on the robot to continuously capture frames of the surrounding environment. These frames are processed in real time using computer vision techniques to identify the presence of a person. Once a person is detected, the robot calculates the person's position and movement within the video frame and responds accordingly to follow their motion.

By combining image processing with real-time motion control, the robot autonomously follows the person without requiring manual intervention.

10 MOBILE APP

Using Android studio, Visual Studio code, Flutter and firebase developed the Mobile App to control the robot like in the same way in GUI manual mode. This was developed to control robot using our mobile phone.

All the users and their commands are stored in the Fire-base real time database. Since this database is cloud based one, we can monitor the command from anywhere.

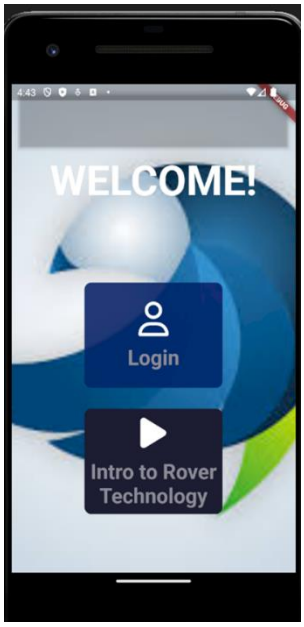


Figure 10.1:login window

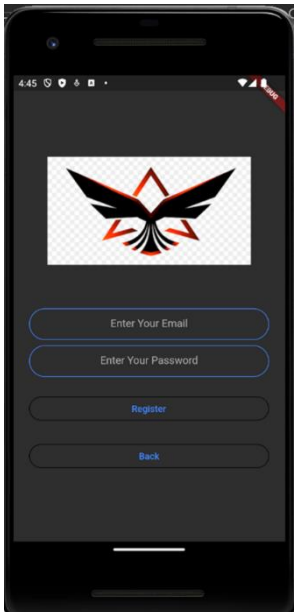


Figure 10.2:authentication window

Figure 10.4:controlling window

Figure 10.3:server communication window

Here when we select the command and confirm that command, navigate to the server communication window then when the send button pressed, command, date & time, user details are stored in the firebase real-time database as showing in the figure below.

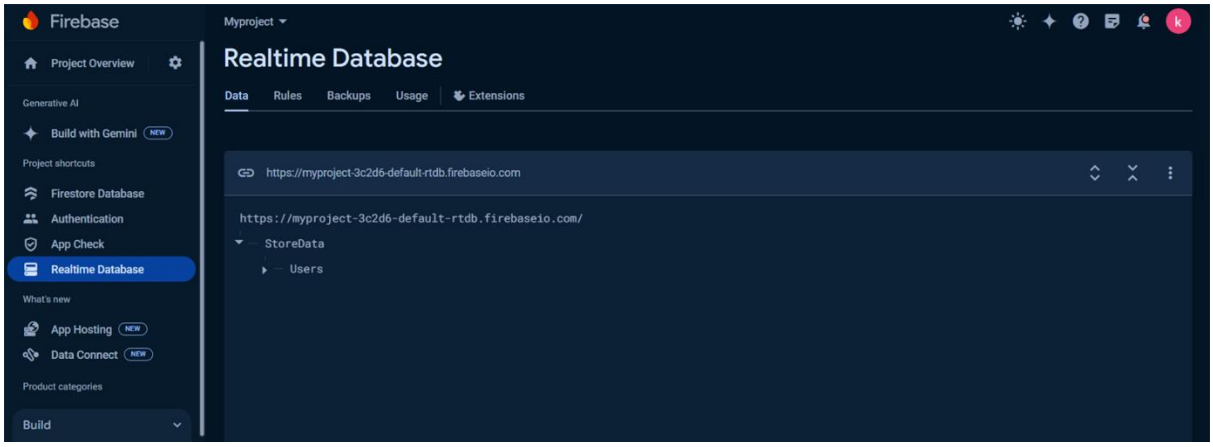


Figure 10.5: firebase real-time database

All the login details also recorded using google authentication facility provide by the firebase.

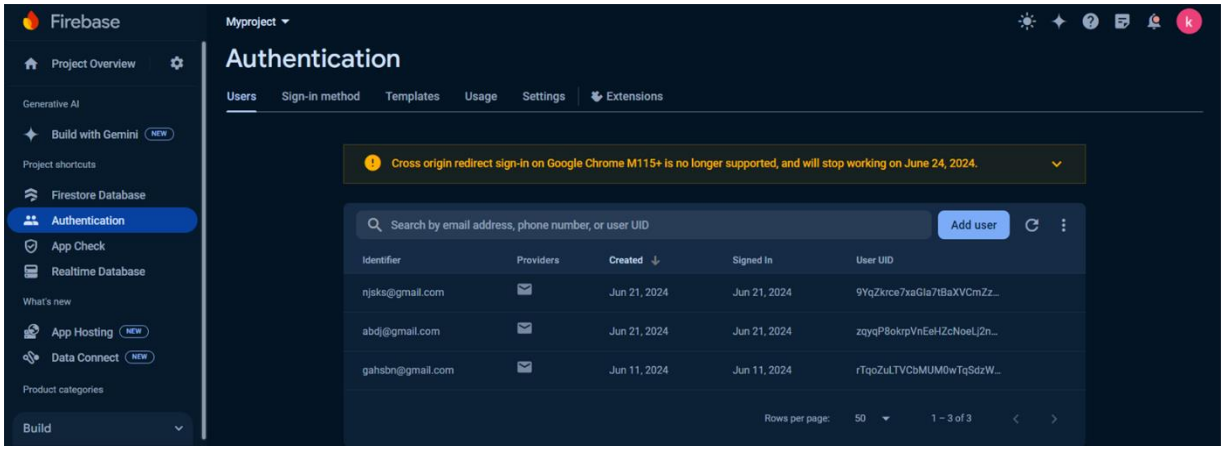


Figure 10.6: login details

In raspberry pi execute the python code to access this database and execute the command to the robot.

Using Android APK Extractor created the Mobile app apk file as well.

11 WEB PAGE

Using HTML, CSS developed the web page and connect it into firebase database. The purpose of developed this web page is if someone want to control the robot using his laptop, he can do that using this web page. This also possible using GUI as well, but GUI run on raspberry pi and user need to access the device. Without doing that simply using web page this can be done.

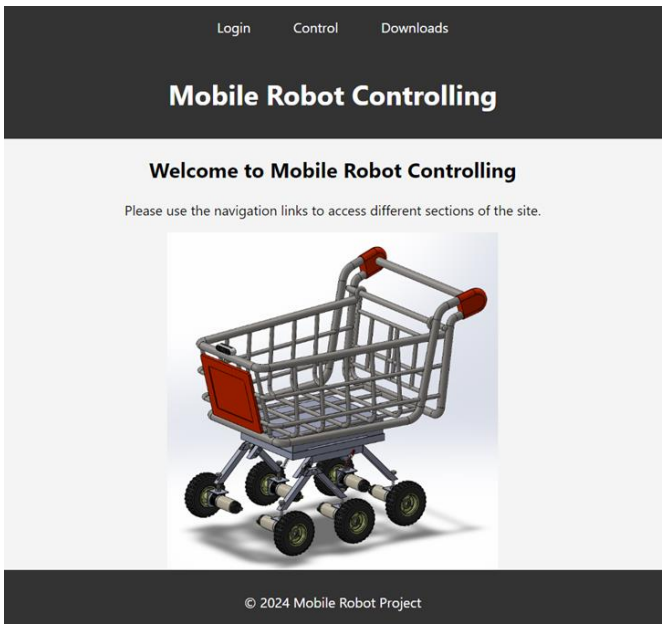


Figure 11.1:front window

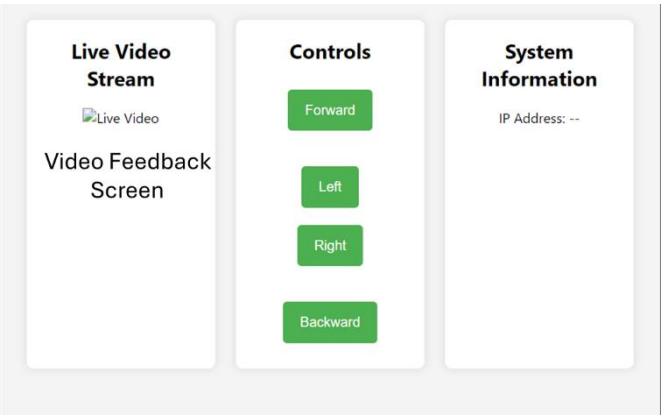


Figure 11.2:controlling window

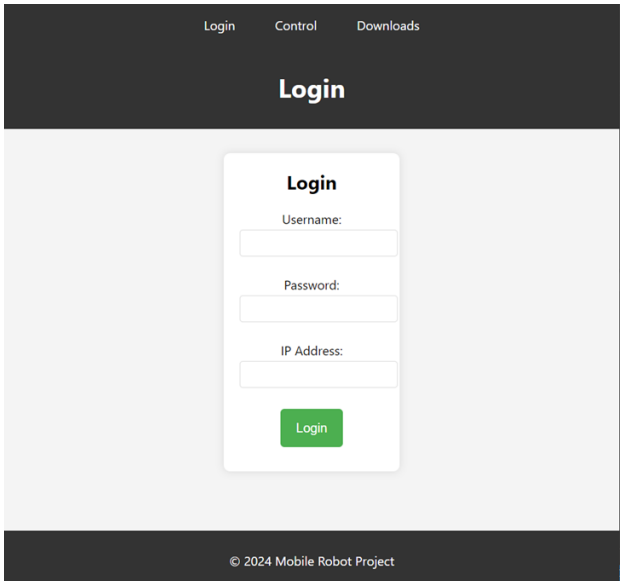


Figure 11.3:login window

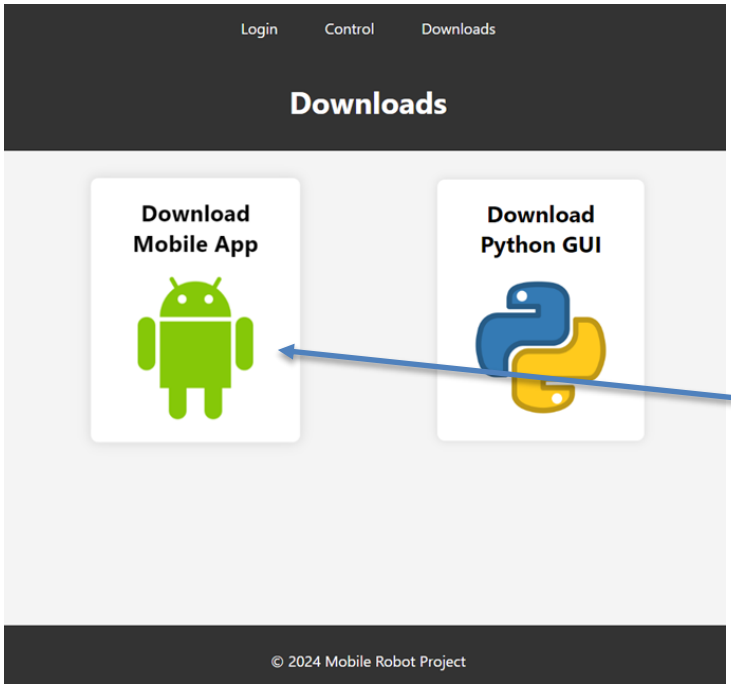


Figure 11.4:download window

There is a separate firebase real-time database for the web page as well.

12 NETWORK CONFIGURATION

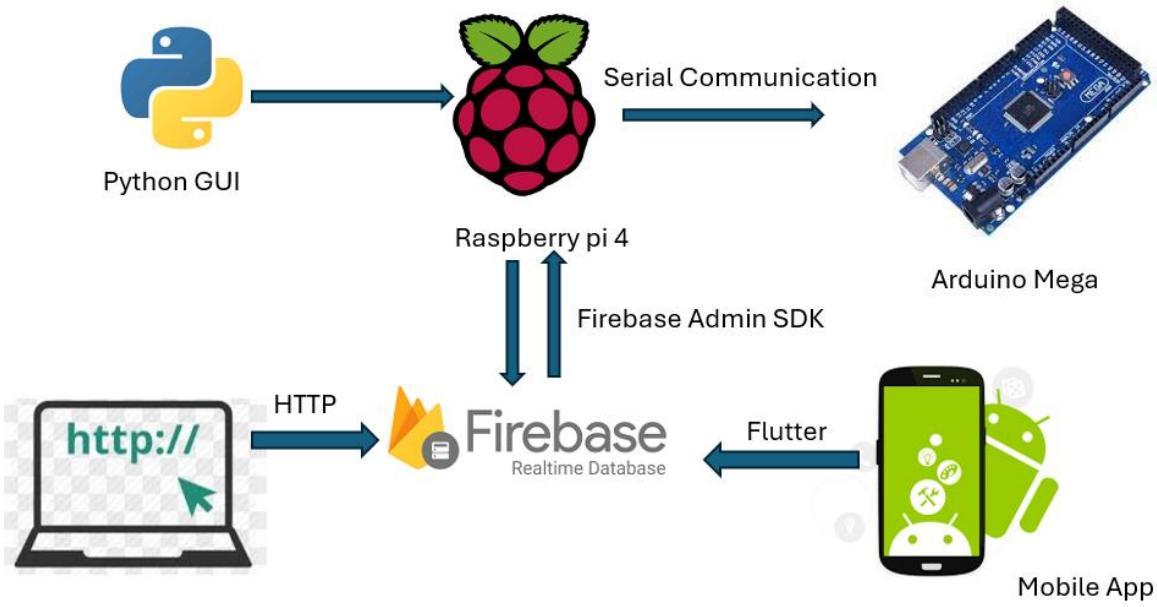


Figure 12.1: Network configuration diagram

As showing in the diagram GUI in directly execute in the raspberry pi and using serial communication it communicates with the Arduino Mega board.

Mobile App and Web page is communicated with raspberry pi through the firebase real-time database.

13 ROS SIMULATION

The main purpose of this simulation is according to the keyboard command my robot should move in virtual environment. To complete this task, I did this ROS simulation in gazebo, all the steps are described in below.

Using SolidWorks to urdf plugin in SolidWorks convert the assembly file of the robot to urdf file.

Then in Ubuntu 20.04 version install ros-neotic and place the robot in gazebo environment. Teleoptwist keyboard plugin is used to send the keyboard command to the robot and custom python script is programmed in order to transfer the angular velocities to the wheel joints of the robot[11].

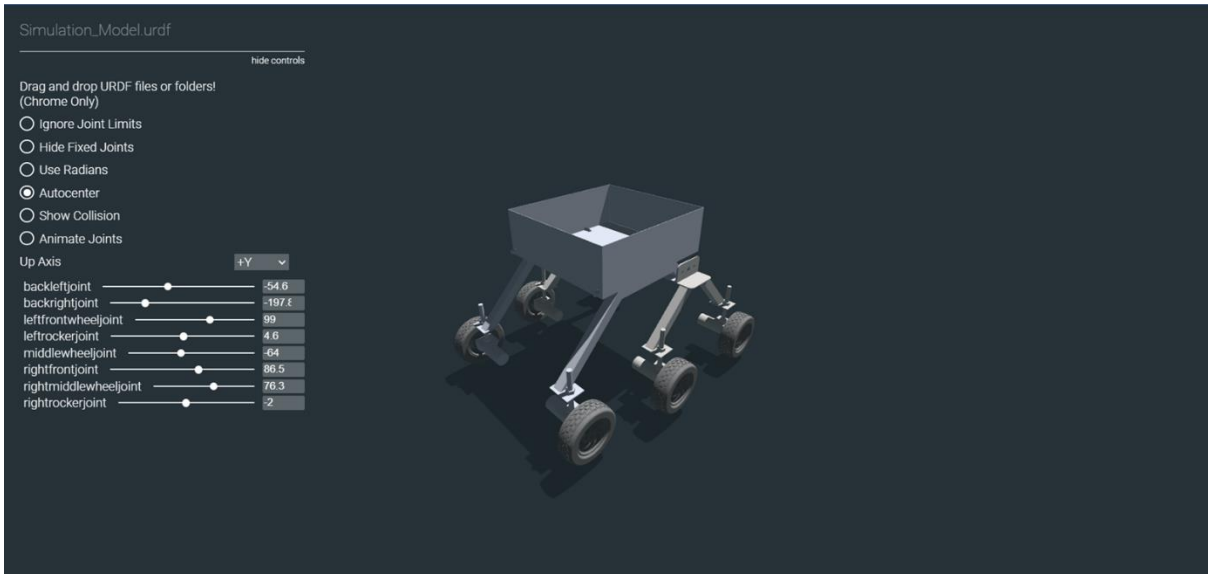


Figure 13.1:solidworks to urdf convert and joints

Positioning robot in gazebo coordinate system and simulate it using keyboard command.

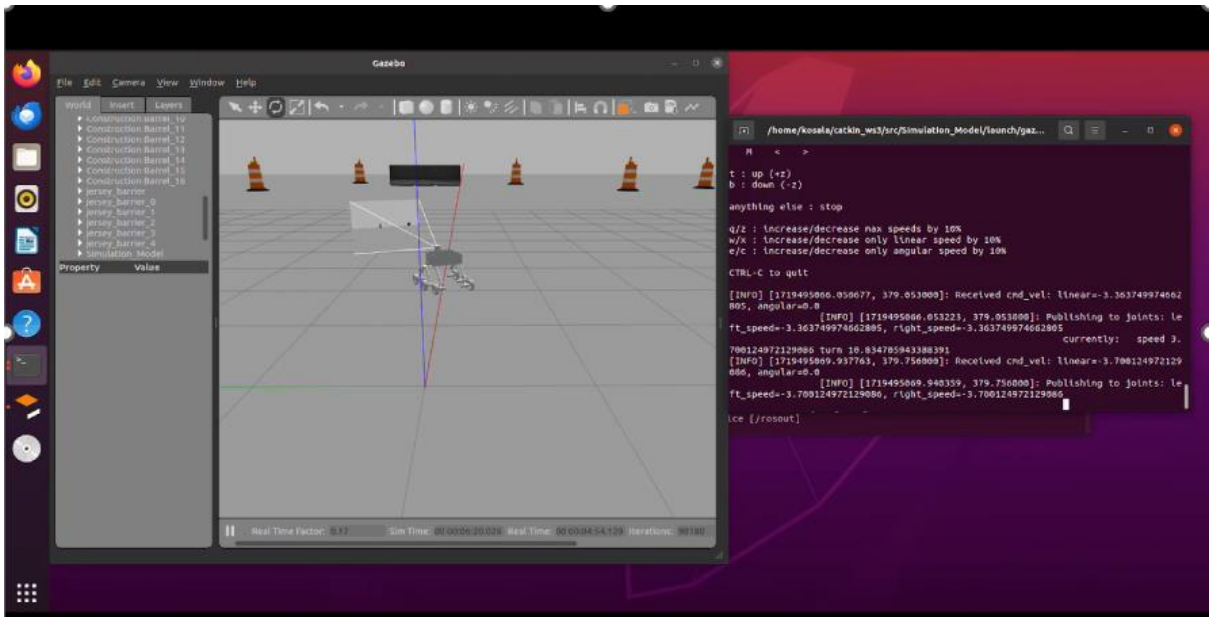


Figure 13.2: simulate in gazebo virtual environment

Then like in the practical scenario, mount the camera and using Rviz visualize the camera feedback as well.

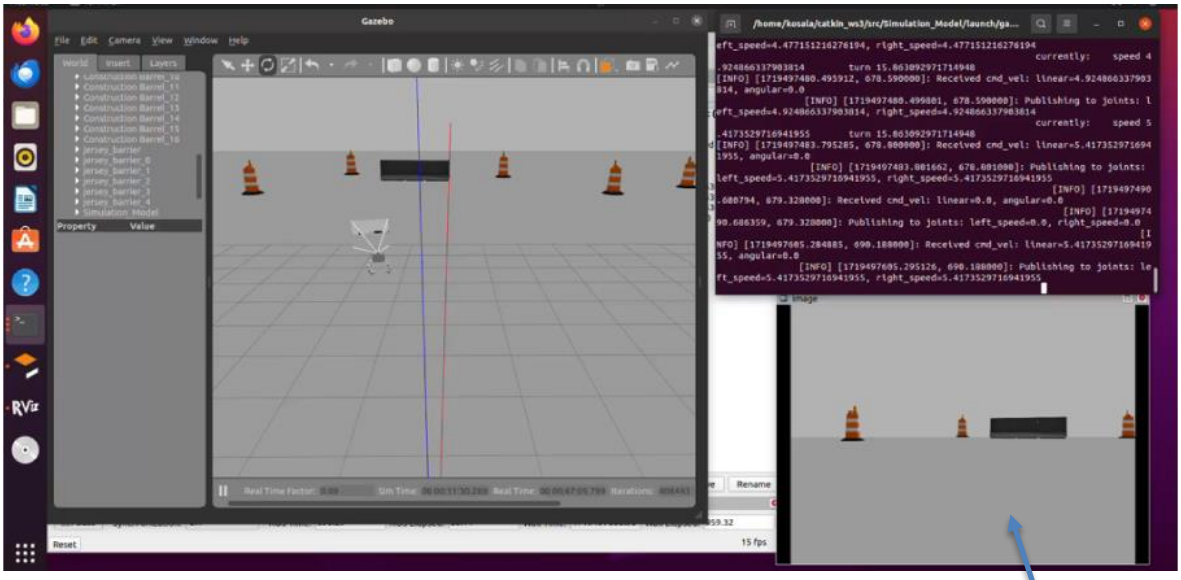


Figure 13.3:camera feedback visualization

Rviz visualization

This simulation is very similar to the manual controlling the robot in practical world. All the things are included in this simulation.

14 ELECTRICAL CIRCUIT

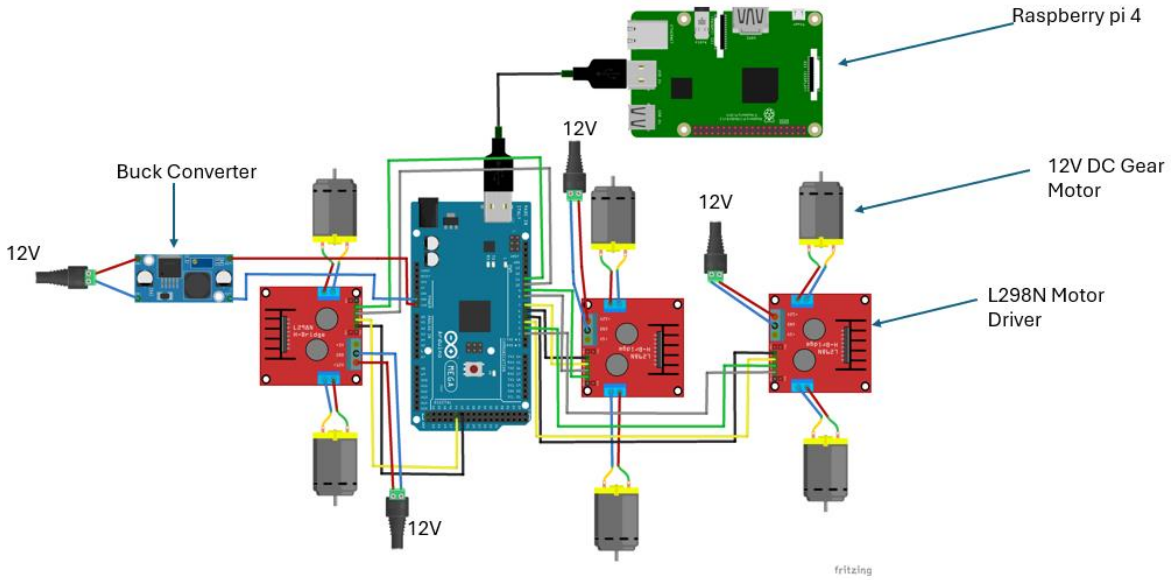


Figure 14.1:electrical circuit schematic diagram

Here I used 12V power supply (12V 10000mah Li-ion battery pack) to power up the Gear motors. Also buck-booster is used to step down the voltage into 5V to power up the Arduino mega board and raspberry pi.

15 PROTOTYPE

Rocker, bogie and chassie frame is developed by using Aluminum box bars.

Control box is design and manufactured by using wood and laser cutting.

Figure 15.2:prototype

Figure 15.1:prototype

Figure 15.4:prototype

Figure 15.3:prototype

16 REFERENCES

[1] R. Hasan, J. Islam, R. M. M. Hasan, and A. N. Paul, "Design of a Vision Based Person Following Robot Well placement optimization View project Design of a Vision Based Person Following Robot." [Online]. Available: https://www.researchgate.net/publication/335892311

[2] S. Hassan, A. F. Khan, M. S. Hassan, and M. W. Khan, "Design and development of human following robot Design and Development of Human Following Robot 1," 2015. [Online]. Available: https://www.researchgate.net/publication/354631715

[3] G. Kuan Chein, U. Tunku, and A. Rahman, "HUMAN FOLLOWING ROBOT USING IMAGE PROCESSING FOR MEDICAL APPLICATION," 2016.

[4] T. Eppenberger, G. Cesari, M. Dymczyk, R. Siegwart, and R. Dubé, *Leveraging Stereo-Camera Data for Real-Time Dynamic Obstacle Detection and Tracking*. [Online]. Available: https://youtu.be/AYjgeaQR8uQ

[5] C. Cosenza, V. Niola, S. Pagano, and S. Savino, "Theoretical study on a modified rocker-bogie suspension for robotic rovers," *Robotica*, vol. 41, no. 10, pp. 2915–2940, Oct. 2023, doi: 10.1017/S0263574723000656.

[6] H. S. Sucuoglu, I. Bogrekci, P. Demircioglu, and O. Turhanlar, "Analysis of Suspension System for 3D Printed Mobile Robot," 2013. [Online]. Available: http://ijamec.atscience.org

[7] A. J. Moshayedi, A. Shuvam Roy, S. K. Sambo, Y. Zhong, and L. Liao, "Review On: The Service Robot Mathematical Model," *EAI Endorsed Transactions on AI and Robotics*, vol. 1, pp. 1–19, Feb. 2022, doi: 10.4108/airo.v1i.20.

[8] H. Fraire, S. Patel, and M. Tirtowidjojo, "OpenCV Person Follower."

[9] "MediaPipe-The Ultimate Guide to Video Processing," 2022. [Online]. Available: https://learnopencv.com

[10] J. E. Grayson, "Pythonand Tkinter Programming Graphical user interfaces for Python programs."

[11] "Mastering ROS for Robotics Programming." [Online]. Available: www.PacktPub.com